Singly Linked List – Insert At Beginning –Space Complexity and Algorithm.

Program:

```
#include <iostream>
#include <stack>
using namespace std;

typedef struct ListNode
{
    int data;
    struct ListNode *next;
} Node;

Node *head;

void createEmptyList(Node **head)
{
    *head = nullptr;
    cout << "Empty list created!!!" << endl;
}

void insertAtBeginning(Node **head, int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr)
    {
        cout << "Error: Memory allocation failed." << endl;
        return;
    }
    newNode->data = data;
    if (*head == nullptr)
    {
        newNode->next = nullptr;
        *head = newNode;
```

```
    }
    else
    {
        newNode->next = *head;
        *head = newNode;
    }
}
int main()
{
   createEmptyList(&head);
    while (1)
    {
        cout << "1. Insert at the beginning" << endl;
        int choice;
        cout << "Enter your choice: ";
        cin >> choice;
        switch (choice)
        {
        case 1:
        {
            int data;
            cout << "Enter the data: ";
            cin >> data;
            insertAtBeginning(&head, data);
            break;
        }
       default:
        {
            cout << "Invalid choice." << endl;
            break;
        }
        }
    }
  return 0;
}
```

Space Complexity

Auxiliary Space:

```
    Node *newNode = (Node *)malloc(sizeof(Node));
```

−The function allocates a single new node using malloc, which is a constant amount of space regardless of the size of the list. This allocation does not depend on the number of elements already in the list.

−The function uses a fixed amount of additional space for local variables (newNode), which also does not depend on the input size.

Auxiliary space is the extra memory used by the function, excluding the input. The insertAtBeginning function performs the following memory operations:

*Declaration of newNode: It declares a single pointer, Node * newNode, which occupies a constant amount of space.*

Memory Allocation: It calls malloc(sizeof(Node)) to allocate memory for exactly one new node. The size of a single node is fixed and does not depend on the number of nodes already in the list.

Since the function only ever creates one new pointer and allocates memory for one new node, the extra space it requires is constant.

Therefore, the auxiliary space complexity is $\Theta(1)$.

Input Space:

−This refers to the space required for the input to the function. In this case, the input space is the space occupied by the head pointer and the data integer. However, since these are passed by reference and value respectively, they do not contribute to additional space complexity

beyond their inherent size.

The input space depends on which convention you follow:

<u>Parameter-Only Definition: O(1)</u>

This definition only considers the space taken by the parameters themselves. The function receives Node **head (a pointer-to-a-pointer) and int data (an integer). Both have a fixed, constant size. Therefore, the input space complexity is O(1).

Parameter-only definition Θ(1)
- `Node **head` **– one pointer-to-pointer (8 bytes on a 64-bit machine)**
- `int data` **– one scalar (4 bytes).**

Both sizes are fixed; they do not depend on the number of nodes already in the list. Therefore parameter-only input space = Θ(1).

<u>Full-Data Definition: Θ(n)</u>

This more common definition includes the entire data structure accessible from the parameters. If the linked list pointed to by head contains n nodes before the function call, the space required to store that list is proportional to n. Therefore, the input space complexity is Θ(n).

- **Everything reachable from the parameters must now be counted.**
- ***head may point to the first node of an existing list of size n.**
- **Each node contains two words (data, next).**

Memory for the list = n × sizeof(Node) = Θ(n) words/bytes.
So the overall input size is Θ(n).

Here we also include everything reachable from the parameters when the call begins. head may point to the first node of an existing list containing n nodes, each of constant size. Total memory already "in play" before the first line of the routine is thus proportional to n. Therefore full-data input space = Θ(n).

Algorithm

$INSBEG(START, ITEM)$

$\left((Node *)malloc\left(sizeof(Node)\right)\right)$ *is availability of space for insertion of Data,*
hence , lets name it AVAIL.

1. $[ALLOCATE\ NEW\ NODE]$ $Set\ NEW := AVAIL.$

2. $[MEMORY\ ALLOCATED?]$ $if\ NEW = NULL, then:$

    $Write:$ `` $Error: Memory\ Allocation\ Failed.$ `` $and\ EXIT$.

    $[End\ of\ If\ structure.]$

3. $Set\ INFO[NEW] := ITEM[Copies\ new\ data\ into\ new\ node].$

4. $[When\ HEAD\ points\ to\ NULL\ i.e.\ LINK\ LIST\ IS\ EMPTY]$

    $If\ START = NULL\ then:$

    $Set\ LINK[NEW] := NULL[Pointer\ of\ NEW\ Node\ i.e. Next\ pointer\ of\ NEW\ node\ assigned\ to\ NULL]$

    $Set\ START := NEW\ [START\ or\ HEAD\ points\ to\ NEW\ NODE's\ address]$

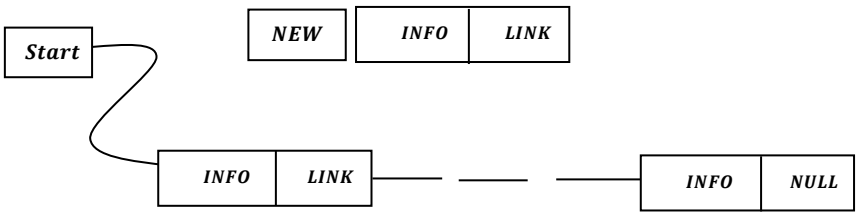
$[When\ HEAD\ doesn't\ points\ to\ NULL\ i.e. LINK\ LIST\ IS\ NOT\ EMPTY]$

$ELSE$

$Set\ LINK[NEW] := START\ [Pointer\ of\ NEW\ Node\ i.e. Next\ pointer\ of\ NEW\ node\ assigned\ to\ HEAD]$

$Set\ START := NEW\ [START\ or\ HEAD\ points\ to\ NEW\ NODE's\ address]$

$[END\ OF\ IF\ STRUCTURE]$

5. $Exit$

DIAGRAM